

Microservicio de Transcripción

El microservicio de Transcripción es un componente esencial dentro de la arquitectura de microservicios del Sistema de Alerta Ciudadana C5. Su función principal es la normalización y procesamiento de señales de audio capturadas por los dispositivos ESP32, convirtiéndolas en texto estructurado que permite la toma de decisiones automatizada.

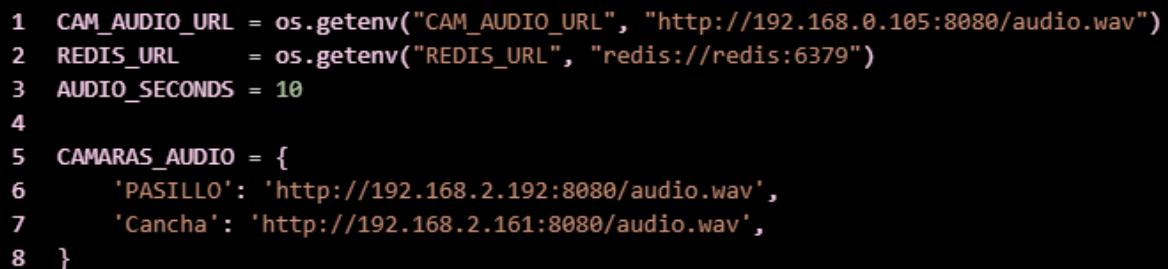
Descripción Técnica y Análisis del Código

El servicio está desarrollado en Python, haciendo uso de contenedores para garantizar la portabilidad.

Análisis de app.py:

El servicio opera bajo un modelo de programación asíncrona, estructurado en los siguientes bloques funcionales:

- **Configuración:** Define los parámetros de conexión para Redis y las URLs de los servicios de audio. Incluye un diccionario CAMARAS_AUDIO que actúa como un enrutador estático para mapear IDs de dispositivos con sus fuentes de audio.



```
1 CAM_AUDIO_URL = os.getenv("CAM_AUDIO_URL", "http://192.168.0.105:8080/audio.wav")
2 REDIS_URL      = os.getenv("REDIS_URL", "redis://redis:6379")
3 AUDIO_SECONDS = 10
4
5 CAMARAS_AUDIO = {
6     'PASILLO': 'http://192.168.2.192:8080/audio.wav',
7     'Cancha': 'http://192.168.2.161:8080/audio.wav',
8 }
```

- **Procesamiento de Audio:** Implementa un cliente aiohttp para descargar el stream de audio en tiempo real y almacenarlo en un archivo temporal tempfile, evitando bloqueos en el hilo principal.

```

1  async def grabar_audio(segundos: int, url: str) -> str:
2      print(f"[Transcription] Grabando {segundos}s de audio desde {url}...")
3      tmp = tempfile.mktemp(suffix=".wav")
4      timeout = aiohttp.ClientTimeout(total=segundos + 5)
5      async with aiohttp.ClientSession(timeout=timeout) as session:
6          async with session.get(url) as resp:
7              data = b""
8              inicio = asyncio.get_event_loop().time()
9              async for chunk in resp.content.iter_chunked(8192):
10                 data += chunk
11                 if asyncio.get_event_loop().time() - inicio >= segundos:
12                     break
13             with open(tmp, "wb") as f:
14                 f.write(data)
15             return tmp

```

- **Transcripción:** Es el motor del servicio. Utiliza el modelo Faster-Whisper configurado con un filtro de voz para optimizar recursos e ignorar silencios, además utiliza un initial_prompt específico para contextos de emergencia, mejorando la precisión en vocabulario de seguridad.

```

1  def transcribir_sync(audio_path: str) -> str:
2      try:
3          segments, _ = model.transcribe(
4              audio_path,
5              language="es",
6              beam_size=5,
7              vad_filter=True,
8              vad_parameters=dict(min_silence_duration_ms=500, speech_pad_ms=400),
9              condition_on_previous_text=False,
10             initial_prompt="Emergencia en el campus universitario. Seguridad, ayuda, guardia, auxilio. ¿Hay alguien ahí?"
11         )
12         texto = " ".join(s.text.strip() for s in segments)
13     except Exception as e:
14         print(f"[Transcription] Error interno en Whisper: {e}")
15         texto = ""
16     finally:
17         if os.path.exists(audio_path):
18             os.unlink(audio_path)
19     return texto
20     segments, _ = model.transcribe(
21         audio_path,
22         language="es",
23         beam_size=1,
24         best_of=1,
25         temperature=0.0,
26     )
27     texto = " ".join(s.text.strip() for s in segments)
28     os.unlink(audio_path)
29     return texto

```

- **Worker y Concurrency:** La función `procesar_alerta` delega la transcripción a un `ThreadPoolExecutor` para asegurar que el sistema no se congele durante la inferencia. La función `escuchar` mantiene un ciclo eficiente de lectura mediante `brpop`.

```
1  async def procesar_alerta(r, alerta: dict):
2      alert_id = alerta.get("alert_id", "unknown")
3      device_id = alerta.get("device_id", "unknown")
4      cam_url = get_cam_audio_url(device_id)
5      print(f"[Transcription] Procesando alerta: {alert_id} | dispositivo: {device_id} | cámara: {cam_url}")
6      t0 = asyncio.get_event_loop().time()
7      try:
8          audio_path = await grabar_audio(AUDIO_SECONDS, cam_url)
9
10         loop = asyncio.get_event_loop()
11         texto = await loop.run_in_executor(executor, transcribir_sync, audio_path)
12
13         elapsed = asyncio.get_event_loop().time() - t0
14         print(f"[Transcription] Listo en {elapsed:.1f}s: {texto}")
15
16         payload = json.dumps({
17             "alert_id": alert_id,
18             "text": texto,
19             "timestamp": datetime.utcnow().isoformat()
20         })
21         await r.publish("channel:transcription", payload)
22     except Exception as e:
23         print(f"[Transcription] Error: {e}")
24
```

```
1  async def escuchar(r):
2      print("[Transcription] Escuchando queue:transcription_input...")
3      while True:
4          try:
5              result = await r.brpop("queue:transcription_input", timeout=5)
6              if not result:
7                  continue
8              alerta = json.loads(result[1])
9              asyncio.create_task(procesar_alerta(r, alerta))
10     except Exception as e:
11         print(f"[Transcription] Error: {e} - reintentando en 2s...")
12         await asyncio.sleep(2)
13
```

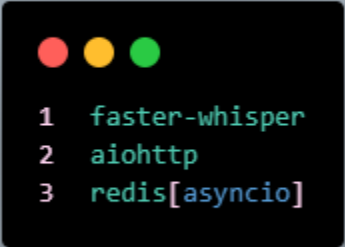
Arquitectura y Despliegue

El servicio se encapsula en un contenedor Docker, garantizando su portabilidad y aislamiento.

Configuración del Dockerfile: El contenedor utiliza una imagen base ligera, instala las dependencias de requirements.txt y expone el entorno necesario para la ejecución del modelo Whisper en modo CPU.

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. It displays the content of a Dockerfile with line numbers from 1 to 15.

```
1 FROM python:3.11-slim
2
3 WORKDIR /app
4
5 # Dependencias del sistema para audio
6 RUN apt-get update && apt-get install -y \
7     ffmpeg \
8     && rm -rf /var/lib/apt/lists/*
9
10 COPY requirements.txt .
11 RUN pip install --no-cache-dir -r requirements.txt
12
13 COPY app.py .
14
15 CMD ["python", "app.py"]
```

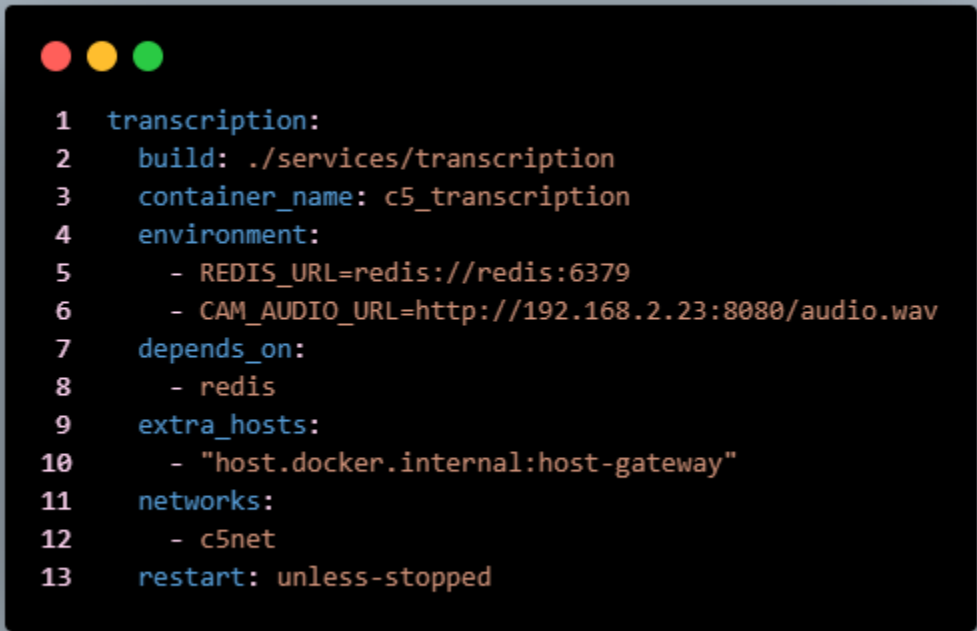
A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. It displays a list of package requirements with line numbers from 1 to 3.

```
1 faster-whisper
2 aiohttp
3 redis[asyncio]
```

Integración y Tolerancia a Fallos

El microservicio es parte de un ecosistema distribuido diseñado para la alta disponibilidad:

- **Persistencia y Cola:** Ante una caída, el servicio depende de la infraestructura de Redis para recuperar tareas pendientes.
- **Balanceo:** El servicio está configurado para ser gestionado por el balanceador de carga del sistema, permitiendo el escalamiento horizontal.



```
1 transcription:
2   build: ./services/transcription
3   container_name: c5_transcription
4   environment:
5     - REDIS_URL=redis://redis:6379
6     - CAM_AUDIO_URL=http://192.168.2.23:8080/audio.wav
7   depends_on:
8     - redis
9   extra_hosts:
10    - "host.docker.internal:host-gateway"
11   networks:
12     - c5net
13   restart: unless-stopped
```